



SAPIENZA
UNIVERSITÀ DI ROMA

Intelligenza Artificiale

Sapienza Università di Roma

Facoltà di Ing. dell'Informazione, Informatica e Statistica

Laurea in Informatica

Prof. Toni Mancini

<https://raise.uniroma1.it/teaching>

Progetto studenti B.1.2 (PS.B.1.2)

Parte B

Agenti Basati su Conoscenza in Logica del
Primo Ordine

Dalle logiche descrittive ai
graph database

1

Introduzione

Questo documento introduce le logiche descrittive, frammenti decidibili della Logica del Primo Ordine che cercano di coniugare espressività ed efficienza dell'inferenza, ed il loro collegamento con i graph database.

Il documento è stato redatto dallo studente Silvio d'Alessio come parte del suo esame di Intelligenza Artificiale, e viene pubblicato senza alcuna garanzia, da parte del docente, circa la sua correttezza e completezza.

Dalle logiche descrittive ai graph database

Silvio D'Alessio

14 Febbraio 2021

1 Logiche Descrittive

1.1 DL vs FOL

Le Description Logics (DL) sono una famiglia di linguaggi formali. Esse permettono di descrivere ontologie e di applicarvi regole di inferenza per dedurre fatti aggiuntivi a partire da quelli esplicitamente descritti nella ontologia d'interesse. Siamo quindi nel campo della cosiddetta “knowledge representation and reasoning”.

Si può notare che esse non sono le sole ad offrire queste funzionalità. La First Order Logic (FOL), ad esempio, anche è in grado di fornirle.

Quindi perché non usare la FOL?

La FOL è una logica general purpose. Essa è dotata di grande potere espressivo, e quindi computazionalmente molto onerosa. Inoltre essa non è decidibile, ma semi-decidibile.

Questo grande potere espressivo tipicamente non è però necessario per le ontologie. Intuitivamente ne è testimone il fatto che, in questo caso, ci sono più modi di esprimere lo stesso concetto in FOL, mentre per i nostri scopi uno è sufficiente.

L'idea è quindi di creare delle logiche che sacrificino parte del potere espressivo della FOL in cambio di migliore performance; in particolare, spesso, recuperando la decidibilità.

Meno è l'espressività necessaria alla descrizione della realtà di interesse, più è la espressività alla quale si può rinunciare e, quindi, maggiore è la performance che può guadagnare. Ed è per questo che esistono molte DL, che si differenziano in base ai costrutti che esse permettono di utilizzare.

1.2 Oggetti delle DL

Le DL rappresentano relazioni tra entità nel dominio di interesse tramite:

- Individui : nomi di oggetti del dominio, ad esempio “Mario” che rappresenta l'individuo chiamato Mario.
- Concetti : insiemi di individui, ad esempio “Femmina”, che rappresenta l'insieme degli individui di genere femminile.

- Ruoli : relazioni binarie tra individui, ad esempio “genitoreDi”, che collega ogni genitore ai suoi figli.

Essi corrispondono rispettivamente a costanti , predicati unari e predicati binari della FOL.

A differenza di un normale database, una ontologia non descrive uno “stato” del mondo di interesse, ma affermazioni, che devono essere vere in esso. Ci possono essere quindi più stati che soddisfano gli assiomi: questi sono cio che l’ontologia vuole rappresentare.

1.3 Box di assiomi

Gli assiomi sono tradizionalmente divisi in tre categorie: ABox, TBox e RBox.

1.3.1 ABox

La ABox contiene assiomi che danno informazioni riguardo gli individui e le loro relazioni con altri individui o concetti. Ad esempio:

- “*Madre(maria)*” fissa il fatto che l’individuo “maria” è una istanza del concetto “Madre”
- “*genitoreDi(maria, mario)*” fissa il fatto che l’individuo “mario” è figlio di “maria”.

Da notare che non vige la Unique Name Assumption, quindi per definire la uguaglianza o disuguaglianza di due concetti è necessario usare appositi assiomi:

- disuguaglianza tra individui: $maria \neq mario$
- uguaglianza tra individui: $maria = mario$

Questi due assiomi vengono utilizzati durante l’ “ontology alignment”, cioè l’operazione di combinare in una unica ontologia le informazioni di uno stesso dominio che sono distribuite in diverse ontologie.

1.3.2 TBox

La TBox contiene assiomi “terminologici”, che descrivono relazioni tra concetti:

- *Concept inclusion*: esprime il concetto di *sussunzione* ad esempio “*Madre* $\sqsubseteq$ *Genitore*”. Questo assioma può essere utilizzato per inferire nuovi fatti riguardanti individui. Esso infatti si comporta come una implicazione logica, fatto che lo rende molto potente, e lo pone alla base di meccanismi per il controllo della consistenza delle ontologie. Vedremo in seguito degli esempi.
- *Concept equivalence*: esprime l’equivalenza di due concetti (classi), ad esempio “*Persona* $\equiv$ *Umano*” , esprime il fatto che tutte le persone sono umani e vice versa.

1.3.3 RBox

La RBox contiene assiomi “relazionali”, ossia assiomi che descrivono relazioni tra i ruoli:

- “*Role inclusion*” e “*Role equivalence*”: simili rispettivamente a “*concept inclusion*” e “*concept equivalence*”.
- “*Role composition*”: analogamente a come si fa con le funzioni, compone due ruoli. Ad esempio “*fratello* $\circ$ *padre* $\sqsubseteq$ *zio*”. Da notare che la composizione deve occupare la parte sinistra e non destra della espressione. Inoltre questo costrutto non può essere usato liberamente, ma solo in determinate condizioni, pena la perdita di decidibilità.
- “*Role disjointness*”: fissa il fatto che i due ruoli non hanno istanze in comune. Ad esempio “*disgiunti*(*genitore*, *figlio*)”.
- Assiomi di riflessività, simmetria ed equivalenza di ruoli.

1.4 Costruttori di ruoli e concetti

Gli assiomi visti finora però non sono molto espressivi. Le DL hanno quindi, inoltre, dei costruttori.

1.4.1 Costruttori booleani di concetti

I costruttori booleani di concetti sono:

- L’intersezione di concetti: ad esempio “*Femmina* $\sqcap$ *Genitore*”, che può essere usata per definire “*Madre*”.
- L’unione di concetti: ad esempio “*Padre* $\sqcup$ *Madre*” che può essere usata per definire “*Genitore*”.
- Complemento di concetti: ad esempio “ $\neg$ *Femmina*”, che può essere usata per definire “*Maschio*”.

I costruttori booleani sono particolarmente utili quando combinati con i concetti “ $\top$ ” e “ $\perp$ ” (si leggono “top” e “bottom”), rispettivamente definiti come il concetto al quale tutti gli individui appartengono e quello al quale nessun individuo appartiene. Ad esempio: Usando “ $\top$ ” si può esprimere che ogni individuo è o maschio o femmina: “ $\top \sqsubseteq \text{Maschio} \sqcup \text{Femmina}$ ”. Usando “ $\perp$ ” si può definire che nessun individuo è sia maschio che femmina: “ $\text{Maschio} \sqcap \text{Femmina} \sqsubseteq \perp$ ”. Tipicamente controllare che un concetto non sussuma “ $\perp$ ”, ossia che esso non sia vuoto, è una tecnica fondamentale per il controllo della consistenza di un concetto.

1.4.2 Restrizioni di ruolo tramite quantificatore

Vediamo infine le restrizioni di ruolo tramite quantificatore, decisamente le più interessanti, che uniscono concetti e ruoli, definendo nuovi concetti. Vediamo ad esempio la relazione tra il concetto "*Femmina*" ed il ruolo "*genitoreDi*":

$$genitoreDiFemmina \equiv \exists genitoreDi.Femmina$$

equivalente alla formula FOL:

$$genitoreDiFemmina(x) \iff \exists y (genitoreDi(x,y) \wedge Femmina(y))$$

Essa fissa il fatto che chi è genitore di una femmina ha almeno una figlia. Analogamente si poteva descrivere chi ha solo figlie femmine con il seguente costruito:

$$genitoreDiSoleFemmine \equiv \forall genitoreDi.Femmina$$

equivalente alla formula FOL:

$$genitoreDiSoleFemmine(x) \iff \forall y (genitoreDi(x,y) \rightarrow Femmina(y))$$

Queste restrizioni possono anche essere usate per definire il dominio di un ruolo, ad esempio:

$$\exists figliaDi.\top \sqsubseteq Femmina$$

esprime il seguente: la classe degli individui tali che sono figlie di qualche individuo è sottoinsieme della classe delle femmine.

Inoltre è possibile definire il rapporto tra il ruolo "*figliaDi*" ed il concetto "*Genitore*":

$$\top \sqsubseteq \forall figliaDi.Genitore$$

che sta a significare che tutti gli individui appartengono alla classe di coloro i quali, se sono in relazione "*figliaDi*" con qualcuno, allora quel qualcuno è un genitore.

Notiamo che la semantica di queste espressioni non è la stessa dei vincoli di un DBMS. Infatti se una "*figliaDi*" si trovasse in relazione con un non genitore, allora un database relazionale lancerebbe un errore di violazione di vincolo, mentre una DL dedurrebbe che quell'individuo deve essere femmina, aggiungendolo al concetto Femmina, e così "aggiustando" il modello.

Ed è proprio questo meccanismo che permette all'operatore di sussunzione (" $\sqsubseteq$ ") di essere usato come un meccanismo di inferenza.

1.4.3 Altre restrizioni di ruolo

Ci sono poi le restrizioni di cardinalità del formato " $\leq N R.C$ "

Che descrivono la classe degli individui che sono in relazione R con al più N individui (distinti) di classe C.

Infine le restrizioni di auto referenzialità locale, usate per descrivere i ruoli riflessivi. Ad esempio: " $\exists parlaCon.self$ ", rappresentante la classe di coloro i quali parlano con se stessi.

1.4.4 Nominali

Altro caso interessante sono i costruttori detti “Nominali” , ossia concetti aventi un solo individuo, spesso definiti con il nome dell’individuo stesso. Essi si esprimono circondando un individuo con parentesi graffe, ad esempio “ $\{mario\}$ ” esprime la concetto comprendente solo “*mario*”. Esse si possono usare per esprimere concetti enumerativi, ad esempio:

$$\{zero\} \sqcup \{uno\} \equiv CifreBinarie$$

Notiamo inoltre come i nominali ci permettano di convertire espressioni della TBox in espressioni della ABox. Ad esempio “*Madre(maria)*” può essere riscritta come

$$\{maria\} \sqsubseteq Madre$$

e “*genitoreDi(maria, mario)*” come

$$\{mario\} \sqsubseteq \exists genitoreDi.\{maria\}$$

Questo mostra come TBox e Abox siano distinzioni solamente convenzionali, senza alcun significato logico profondo.

1.4.5 Costruttori di ruoli

Abbiamo anche il costruttore del ruolo inverso: “*maritoDi* $\equiv$ *moglieDi*⁻”

Inoltre, analogamente ai concetti “ $\top$ ” e “ $\perp$ ” , abbiamo il ruolo universale, che associa tutti gli individui a tutti gli altri, ed il suo complemento: il ruolo vuoto. Quest’ultimo, a differenza del primo, è zucchero sintattico per un qualunque ruolo “*R*” tale che “ $\top \sqsubseteq \neg \exists R.\top$ ”. Allo stesso modo, transitività , irreflessività , simmetria , antisimmetria non hanno bisogno di costruttori primitivi, ma possono essere espresse usando i costrutti precedentemente descritti.

2 La logica descrittiva “SROIQ”

Tutti i costrutti che abbiamo visto fin ora formano una DL chiamata “*SROIQ*”.

Essi però non possono essere usati liberamente, pena la perdita di un algoritmo decidibile per fare inferenza su ontologie costruite con essi. Pertanto, ci sono delle restrizioni strutturali, qui omesse per brevità, che devono essere rispettate per garantire la decidibilità.

Data una interpretazione, una ontologia si dice consistente se essa ha almeno un almeno un modello. Un assioma è conseguenza di una ontologia se esso è verificato in tutti i modelli di essa, anche se ciò, non è stato esplicitamente dichiarato! Questa è una differenza con i database relazionali, che , esprimendo un solo stato del mondo, verificano una affermazione se e solo se essa concorda con il loro modello corrente. In un certo senso, quindi, le ontologie sono più “intelligenti”.

Avendo noi definito un frammento di FOL, la funzione di interpretazione non è diversa da quella di FOL. Ecco quindi qui riassunte sintassi e semantica di SROIQ, dove I è la funzione di interpretazione.

SROIQ: assiomi [1]

	Syntax	Semantics
<i>ABox</i> :		
concept assertion	$C(a)$	$a^I \in C^I$
role assertion	$R(a, b)$	$\langle a^I, b^I \rangle \in R^I$
individual equality	$a \approx b$	$a^I = b^I$
individual inequality	$a \neq b$	$a^I \neq b^I$
<i>TBox</i> :		
concept inclusion	$C \sqsubseteq D$	$C^I \subseteq D^I$
concept equivalence	$C \equiv D$	$C^I = D^I$
<i>RBox</i> :		
role inclusion	$R \sqsubseteq S$	$R^I \subseteq S^I$
role equivalence	$R \equiv S$	$R^I = S^I$
complex role inclusion	$R_1 \circ R_2 \sqsubseteq S$	$R_1^I \circ R_2^I \subseteq S^I$
role disjointness	$\text{Disjoint}(R, S)$	$R^I \cap S^I = \emptyset$

SROIQ: costruttori [1]

	Syntax	Semantics
<i>Individuals</i> :		
individual name	a	a^I
<i>Roles</i> :		
atomic role	R	R^I
inverse role	R^-	$\{\langle x, y \rangle \mid \langle y, x \rangle \in R^I\}$
universal role	U	$\Delta^I \times \Delta^I$
<i>Concepts</i> :		
atomic concept	A	A^I
intersection	$C \sqcap D$	$C^I \cap D^I$
union	$C \sqcup D$	$C^I \cup D^I$
complement	$\neg C$	$\Delta^I \setminus C^I$
top concept	$\top$	Δ^I
bottom concept	$\perp$	$\emptyset$
existential restriction	$\exists R.C$	$\{x \mid \text{some } R^I\text{-successor of } x \text{ is in } C^I\}$
universal restriction	$\forall R.C$	$\{x \mid \text{all } R^I\text{-successors of } x \text{ are in } C^I\}$
at-least restriction	$\geq n R.C$	$\{x \mid \text{at least } n R^I\text{-successors of } x \text{ are in } C^I\}$
at-most restriction	$\leq n R.C$	$\{x \mid \text{at most } n R^I\text{-successors of } x \text{ are in } C^I\}$
local reflexivity	$\exists R.\text{Self}$	$\{x \mid \langle x, x \rangle \in R^I\}$
nominal	$\{a\}$	$\{a^I\}$

where $a, b \in N_I$ are individual names, $A \in N_C$ is a concept name, $C, D \in \mathbf{C}$ are concepts, $R \in \mathbf{R}$ is a role

3 Frammenti di SROIQ

Le differenti DL si differenziano per i costruttori e assiomi che permettono di utilizzare. Essi sono spesso un sottoinsieme di SROIQ. Per esempio, ALC è un frammento di SROIQ che ha la RBox vuota e permette solamente intersezione, unione e complemento di concetti, oltre ai costrutti con quantificazione esistenziale e universale che abbiamo visto. L'estensione di ALC con ruoli transitivi è tradizionalmente denotata con la lettera "S". Altre lettere denotano altre particolari caratteristiche di una DL, ed esse vengono tradizionalmente aggiunte al nome della DL, rendendo così il nome di essa una sorta di documentazione della logica stessa: è questa nomenclatura che ha dato origine al nome SROIQ, dove la "S" ha il significato sopra descritto.

Diverse DL fanno diversi trade off tra potere espressivo e costo computazionale delle inferenze. Esse sono specializzate in base a diverse esigenze. Vediamo di seguito come il linguaggio "OWL" fa uso di alcune di esse.

4 OWL: Web Ontology Language

Il Web Ontology Language (OWL) è un W3C standard simile alle DLs, la differenza più significativa è nella terminologia usata: classi anziché concetti e proprietà anziché ruoli. Esso è stato introdotto nel 2004 e rimpiazzato da OWL 2 nel 2009. Di seguito considereremo solamente OWL 2.

OWL è basato su SROIQ, ed esiste in diverse varianti, che si distinguono in base alla DL utilizzata.

4.1 OWL: versioni la cui inferenza è polinomiale

Le varianti di OWL la cui inferenza richiede tempo polinomiale sono le versioni più ridotte, ossia quelle che restringono ulteriormente il potere espressivo di *SROIQ* :

- OWL EL: polinomiale, basato sull'omonima DL
- OWL QL: addirittura meno che polinomiale, basato sulla DL "DL-lite"
- OWL RL: descrive dei Description Logic Programs (DLP), una famiglia di DLs sintatticamente ristretta di modo che gli assiomi possono essere interpretati come clausole Horn in logica del primo ordine (ma senza funzioni).

4.2 OWL DL

OWL DL è una variante che sacrifica la polinomialità: essenzialmente SROIQ con zucchero sintattico. Basandosi su SROIQ e rispettando le restrizioni strutturali sopra citate, l'inferenza di OWL DL è decidibile, sebbene esponenziale.

Inoltre OWL DL fa uso di "datatype literals", ossia concetti con interpretazione predefinita (ad esempio i booleani). La distinzione tra individui datatype

ed altri individui (detti "astratti"), è simile a quella tra valori di tipo primitivo e non in Java. In OWL DL questa distinzione è manifesta nei nomi dei costrutti: essi cominciano con "Data" se fanno riferimento ad individui con interpretazione predefinita, o con "Object" se fanno riferimento ad individui astratti. Un esempio per tutti: "DataIntersectionOf" e "ObjectIntersectionOf".

Infine, ecco un esempio di come OWL DL esprime *SROIQ*:

$$Madre \equiv Femmina \sqcap Genitore$$

viene scritto come

EquivalentClass(Madre ObjectIntersectionOf(Femmina Genitore)) .

4.3 OWL FULL

OWL non è però nato solo come implementazione di una DL, ma anche come estensione del Resource Description Framework (RDF), un linguaggio di modellazione la cui espressività è comparabile a quella delle ABox.

Nonostante OWL ed RDF abbiano simili casi d'uso, le loro semantiche formali non coincidono.

Se combiniamo i due otteniamo OWL FULL, che, come OWL DL, si basa su SROIQ, ma a causa dell'integrazione con l'RDF esso perde la decidibilità. L'inferenza con esso è pertanto indecidibile.

Tipicamente questa è la versione di OWL che si usa, espressa come Triplestore grazie al formato RDF, che ora vediamo.

5 RDF: Resource Description Framework

L'RDF è lo standard alla base del formato Triple store. Un Triple store è un insieme di terne (ordinate) su un vocabolario, i cui campi sono chiamati rispettivamente : soggetto, predicato e oggetto.

Intuitivamente, esso prova ad imitare brevi frasi del linguaggio naturale come "Mario ama Maria" o "Maria ha 100 euro", risultando quindi sorprendentemente intuitivo alla lettura. Esso può anche ricordare un oggetto di Java: come ad un oggetto di java è associato un mapping tra campo e valore, allo stesso modo ad un soggetto è associato un mapping prediato-oggetto... se non fosse che a differenza di Java ad ogni chiave possono essere associati più valori. In altre parole, il predicato della tripla non è una funzione, ma una relazione!

Formalmente, Ignorando i "blank node" (praticamente zucchero sintattico), ogni tripla è composta da ¹, nell'ordine:

- soggetto: uno URI
- predicato: uno URI

¹Nota: più precisamente, non si usano URI ma la loro generalizzazione : gli IRI, che supportano l'uso di caratteri non ASCII.

- oggetto: uno URI o un valore primitivo.

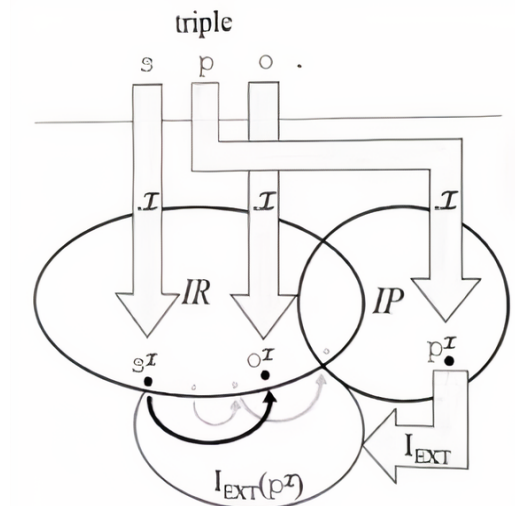
Tutto è quindi rappresentato da uno URI o un valore primitivo. Il fatto che si usino gli URI permette di fare riferimento più volte allo stesso soggetto e quindi poter definire tripla per tripla le sue proprietà.

Per interpretare le triple abbiamo bisogno di :

- IR : l'insieme delle "risorse" del quale le triple "parlano", corrispondente a quello che in FOL viene detto "dominio". Esso contiene qualunque termine compaia come soggetto o oggetto, inclusi valori primitivi.
- IP : Un insieme di (nomi di) proprietà.
- I_{ext} : Una funzione che associ ogni nome di proprietà ad una relazione binaria tra le risorse, rappresentazione equivalente ad un grafo diretto.
- I_s : Una funzione che associ ad ogni URI usato nelle triple la risorsa, o la proprietà, ad esso corrispondente.
- I_l : Una funzione che associ ad ogni valore primitivo la corrispondente risorsa che esso rappresenta, solo se essa ha un datatype definito (altrimenti viene gestita da LV).
- LV : il sottoinsieme delle risorse (IR) che sono valori primitivi, solo se essi hanno datatype NON definito (altrimenti gestiti da I_l).

Possiamo allora definire una funzione di interpretazione che associa ad ogni valore primitivo la corrispondente risorsa e ad ogni URI la corrispondente risorsa o proprietà.

una illustrazione della funzione di interpretazione delle triple [2]



In pratica stiamo definendo un grafo diretto, con archi etichettati, equivalente all'insieme di grafi diretti associati alle proprietà, e stiamo salvando

nell'etichetta dell'arco il nome del grafo (rappresentate la proprietà) dal quale l'arco "proviene".

Le triple possono essere serializzate in vari formati, ad esempio XML e JSON, o anche sintassi testuali ad-hoc come il formato "Terse RDF Triple Language", meglio noto come "Turtle".

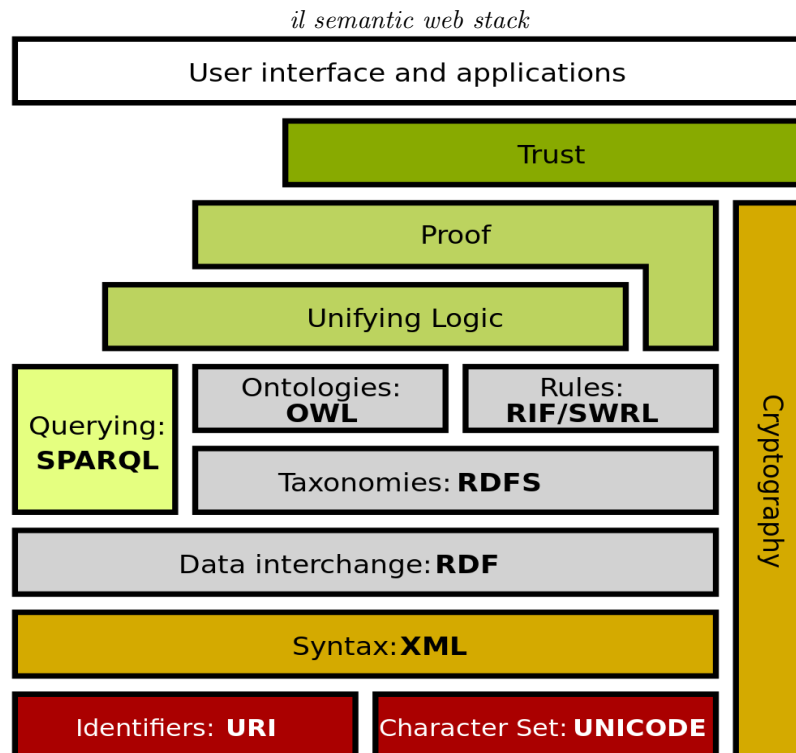
6 Semantic Web Stack

Abbiamo ora una visione quasi completa della cosiddetta "semantic web stack". Mancano soltanto SWRL e SPARQL.

SWRL è il "Semantic Web Rule Language". Esso è un tentativo di coniugare OWL ed inferenza Horn. A differenza di OWL RL, SWRL non compromette l'espressività di OWL in cambio di performance. OWL RL è infatti l'intersezione del potere espressivo di *SROIQ* ed inferenza Horn ed è quindi un sottoinsieme di *SROIQ*, che (con i dovuti accorgimenti, che OWL RL segue) è decidibile. SWRL ne è invece l'unione, ed è quindi quindi un sovrainsieme di *SROIQ*, quindi esso può raggiungere, e raggiunge, l'indecidibilità.

Infine, SPARQL è un linguaggio per eseguire query su Triple-store simile all'SQL.

Abbiamo, finalmente, una visione completa del "semantic web stack".



7 Applicazioni e problemi

La semantic web è nata con lo scopo di “fornire un framework comune che permetta la condivisione ed il riuso di dati tra applicazioni, aziende e comunità”.

Funzionale a questo scopo fu la scelta di usare gli URI per identificare univocamente i vocaboli delle triple. Questo permette di scongiurare l’insorgere di ambiguità nel caso si voglia unire due Triple-store, che diventa così tanto semplice quanto unire i due insiemi di triple.

Quando però si utilizza il semantic web stack per definire una ontologia, sorgono altri problemi. Unire due Triple-store senza errori non è sufficiente ad effettuare un’ “ontology alignment”. Infatti, ci sono ancora problemi da risolvere, ad esempio:

- due diverse ontologie possono assegnare due URI (o nomi) diversi ad uno stesso concetto.
- Se il concetto è derivato (cioè espresso in funzione di altri) esso può essere rappresentato equivalentemente con due grafi non isomorfi.
- Un concetto può essere primitivo in una ontologia e derivato in un’altra

A livello pratico sorgono ulteriori problemi quando le triple si avvicinano troppo al linguaggio naturale: chi le progetta rischia di dimenticare le basi matematiche della ontologia e di incorrere in errori concettuali.

Ad esempio, OWL ha un costrutto chiamato “maxCardinality” analogo alle restrizioni di cardinalità che abbiamo visto in *SROIQ*. Immaginiamo di usarlo per dichiarare che ogni moglie ha al più un marito. In questo caso è possibile che ad ella vengano associati due mariti: in qual caso la semantica imporrà che i due mariti vengano considerati essere lo stesso individuo, nessun errore verrà lanciato, l’ontologia non risulterà inconsistente, perchè l’operatore di sussunzione non fa vincoli, ma inferenze! Un utente non esperto potrebbe ingenuamente scambiare per un vincolo di cardinalità di un database relazionale, che invece lancerebbe un errore, ed utilizzarlo impropriamente.

Analogamente, se un vincolo MinCardinality indica che esiste almeno un marito, allora anche in caso nessun marito sia stato definito, il vincolo viene semplicemente ignorato perchè , grazie base alla open world assumption, un marito potrebbe esistere ma non essere registrato, e potrebbe essere aggiunto in seguito.

Purtroppo spesso sono gli esperti del dominio di interesse a scrivere le ontologie, e non matematici.

Quindi, sia a causa di problemi teorici che pratici, per effettuare l’ontology alignment di due ontologie, la comprensione da parte di esse di un umano è ancora necessaria.

L’idea di usare delle triple, o meglio un grafo con archi etichettati, non è però esclusiva alle ontologie. Essa è stata applicata anche nel campo dei database convenzionali, dando luogo ai così detti “Graph Database”. Il più diffuso dei quali è “Neo4j”.

8 Neo4J Graph Database

Anche Neo4j infatti rappresenta nodi connessi da archi etichettati. Ma a differenza di un semplice Triple-store , sia i nodi sia gli archi hanno associati un loro albero, simile ad un albero JSON, che mantiene quello che OWL chiama “DatatypeProperties”, ossia archi che collegano l’oggetto in questione ad un valore primitivo. E’ una scelta utile dal momento che il valore primitivo non e’ referenziato da altri nodi, e conviene quindi risparmiare l’arco ed il nodo necessari al suo accesso e porre il valore direttamente ”all’interno” dell’unico nodo che ne fa uso, diminuendo così la dimensione del grafo.

Neo4j inoltre permette di definire vincoli, indici , ed altri strumenti tipici dei database, la cui semantica è familiare all’utente medio.

Esso è senza dubbio più user friendly e meno logicamente complesso di Triple-stores basate sull’RDF, ed offre ad essi una valida alternativa per chiunque sia interessato ad usare un grafo per rappresentare il proprio dominio di interesse.

Bibliografia

- [1] M. Krötzsch, F. Simancik, and I. Horrocks, “A description logic primer,” *Perspectives on Ontology Learning*, vol. 18, 01 2012.
- [2] P. Hitzler, M. Krötzsch, and S. Rudolph, *Foundations of Semantic Web Technologies*. Chapman and Hall/CRC Press, 2010.